

ChemEST Java

Relatório técnico de entrega acadêmica | 30/09/2026

1. Introdução

O ChemEST Java adapta etapas de preparação e análise do fluxo apresentado por Han et al. (2025), no artigo “Integrating Electronic Structure Theory Calculations with Python and Cheminformatics Tools”. O problema abordado é organizar representações moleculares, arquivos de entrada e resultados de estrutura eletrônica em uma aplicação acadêmica com responsabilidades separadas.

O objetivo desta entrega é demonstrar o fluxo implementado, preservar o código existente e produzir entradas, saídas e evidências reproduzíveis. Nenhum cálculo quântico foi executado para os exemplos desta entrega. O arquivo OUT é sintético e a geometria de etanol é uma construção didática, sem otimização Gaussian.

2. Trabalho original

O artigo integra programação Python e ferramentas de quimioinformática ao ensino de estrutura eletrônica. RDKit participa da manipulação de representações e estruturas moleculares; Gaussian e PySCF são os programas que realizam os cálculos quânticos. SMILES descreve a conectividade molecular, mas não constitui, por si só, uma geometria tridimensional.

Na análise de orbitais, HOMO designa o orbital ocupado de maior energia e LUMO o orbital virtual de menor energia. O gap orbital é a diferença entre essas energias. Ele não deve ser interpretado automaticamente como uma energia de excitação experimental.

O ChemEST cobre um subconjunto do fluxo educacional: usa CDK em Java para SMILES e representação 2D, importa XYZ e prepara arquivos Gaussian. Não reproduz todas as ferramentas ou exemplos científicos do artigo. Fonte: <https://doi.org/10.1021/acs.jchemed.4c01171>.

3. Proposta Java

O fluxo pretendido é:

SMILES → representação 2D | XYZ fornecido separadamente → GJF → Gaussian externo → OUT → ChemEST → HOMO/LUMO/gap.

A representação 2D não é convertida automaticamente em geometria 3D. As coordenadas são importadas de XYZ; a preparação do GJF e a análise do OUT são operações independentes. Gaussian/PySCF permanecem externos à aplicação, que não implementa DFT ou Hartree-Fock.

Nesta demonstração, a sequência não representa uma execução científica contínua: o GJF de etanol foi realmente gerado, mas não submetido ao Gaussian. Os resultados da análise vêm de um OUT sintético e não correspondem a energias calculadas para esse etanol.

4. Arquitetura

Pacote	Responsabilidade
model	Atom3D, MoleculeRecord, CalculationConfig, OrbitalResult e AnalysisRecord; dados e invariantes
service	SMILES/CDK, importação XYZ e coordenação de análise e lote
io	Leitura CSV/OUT; escrita GJF, BCF e CSV de resultados
report	Relatório HTML com escape de textos
ui	MainFrame, cinco abas Swing, eventos e SwingWorker
experimental	Servidor local da interface web opcional

App inicia Swing por padrão. MainFrame coleta entradas e exibe resultados; GaussianOutputParser contém o pareamento dos orbitais, e OrbitalResult calcula os gaps. Os scripts de entrega ficam em docs/validation, fora de src/main e sem dependências adicionais de produção.

O projeto utiliza Java 21, Maven, CDK 2.13, Commons CSV 1.14.1 e JUnit 5.13.4. O pom.xml foi preservado.

5. Funcionalidades implementadas

A interface principal possui as abas Molécula, Entrada Gaussian, Lote, Análise e Log. Permite validar SMILES e desenhar moléculas em 2D, importar XYZ, configurar o cálculo, gerar GJF, importar CSV com NUM/SMILES e preparar lotes e BCF. A análise aceita saídas textuais com os padrões Alpha reconhecidos e exporta CSV e HTML. Operações maiores usam SwingWorker; falhas individuais de análise/lote são apresentadas sem interromper todos os registros.

A entrega distingue três categorias:

Categoria	Arquivos principais
Entradas demonstrativas	data/exemplos/etanol.xyz, 1.xyz, moleculas.csv e gaussian_sample.out
Saídas do ChemEST	resultados/exemplo.gjf, exemplo.bcf, resultados.csv e relatorio.html
Documentação e evidências	Este relatório em Markdown/PDF, comparacao_python_java.csv e docs/validation

A geometria tem fórmula C₂H₆O, nove átomos e hidrogênios explícitos. Foi construída com arranjo tetraédrico idealizado nos carbonos e distâncias de exemplo: C-C 1,52 Å, C-O 1,43 Å, C-H 1,09 Å e O-H 0,96 Å. É um modelo didático não otimizado; as distâncias são parâmetros de construção, não resultados medidos. O importador FileCoordinateProvider aceitou o XYZ.

1.xyz é uma cópia exata de etanol.xyz para o identificador NUM=1. Não existe 2.xyz: o benzeno do CSV não recebeu geometria inventada. A execução do serviço de lote produziu um sucesso e uma falha esperada pela ausência dessa geometria.

6. Adaptação do parser Python

A referência Python desta validação é uma implementação independente de `extract_HOMO_LUMO`, escrita conforme a lógica solicitada no roteiro. Ela lê o conteúdo em memória e busca blocos ocupados seguidos de uma linha virtual, selecionando o último par completo. Não é uma cópia autenticada do código original dos autores; a comparação demonstra equivalência na amostra utilizada, não em todos os arquivos possíveis.

GaussianOutputParser usa `BufferedReader` e processamento linha por linha em ISO-8859-1. Atualiza o HOMO temporário a cada linha Alpha ocupada e salva o par ao encontrar a primeira linha virtual correspondente. O HOMO é o último valor ocupado; o LUMO é o primeiro virtual. Linhas virtuais de continuação não substituem esse LUMO.

Um bloco ocupado incompleto no final não substitui um par completo anterior. Se não houver nenhum par completo, ocorre `GaussianParseException`; erros de arquivo usam `IOException`. Apenas Alpha é tratado.

$$\Delta E = E_LUMO - E_HOMO$$

$$\text{gapHartree} = \text{LUMO} - \text{HOMO}$$

$$\text{gapEv} = \text{gapHartree} \times 27.211386245988$$

$$1 \text{ Hartree} = 27.211386245988 \text{ eV}$$

7. Melhorias realizadas

A leitura em fluxo evita armazenar todo o texto do OUT, reduzindo essa parcela de uso de memória em comparação com a referência que carrega o conteúdo integralmente. Isso não constitui um benchmark de memória ou velocidade, nem afirma que Java seja mais rápido.

A organização orientada a objetos separa modelo, serviços, arquivos e interface. Validações de campos, coordenadas e números finitos, exceções compreensíveis, interface Swing, conversão automática, exportações e testes automatizados tornam o fluxo mais verificável. Esses recursos foram confirmados na implementação Java; não se presume que todos estivessem ausentes no trabalho Python original.

8. Validação

GaussianOutputParser e a referência Python 3.14.6 foram executados sobre o mesmo arquivo data/exemplos/gaussian_sample.out. O runner Java registrou o SHA-256 da entrada e o script Python conferiu esse hash antes da comparação. A amostra preserva as linhas orbitais do recurso de testes e acrescenta identificação explícita: AMOSTRA SINTÉTICA PARA TESTES.

Grandeza (Hartree)	Python	Java	Diferença absoluta
HOMO	-0.250000	-0.250000	0.000000
LUMO	-0.050000	-0.050000	0.000000
Gap	0.200000	0.200000	0.000000

A tolerância aplicada foi 1e-8 Hartree. Todas as diferenças ficaram abaixo do limite; status da comparação: OK. A conversão Java resultou em 5.4422772491976 eV. O CSV comparativo e o JSON de evidências foram produzidos pelo script executado, sem resultados preenchidos manualmente.

GaussianInputWriter gerou exemplo.gjf com SMILES CCO, XYZ importado e CalculationConfig.defaults(): B3LYP/6-31G(d), Opt, carga 0, multiplicidade 1, 2GB e dois processadores. ResultCsvWriter e HtmlReportWriter produziram as exportações a partir da análise Java. O HTML identifica o OUT sintético e a configuração como referência, não como metadado extraído da saída.

GaussianBatchWriter gerou o BCF a partir do GJF real. Como a classe grava caminhos absolutos, o runner realizou um único pós-processamento documentado: substituiu o prefixo da pasta local por caminhos relativos à pasta do BCF. Nenhuma energia ou configuração foi alterada. O exemplar entregue exige a pasta resultados como base; para uso real, regenere pelo aplicativo no ambiente de destino.

BCF gerado estruturalmente pelo ChemEST Java; execução no Gaussian não validada.

9. Testes

Os testes existentes abrangem o parser, último par completo, múltiplas linhas, arquivos incompletos, valores inválidos, conversão com tolerância 1e-8, SMILES válido/aromático/inválido, configuração, XYZ, GJF, leitura CSV, lote, BCF, exportações CSV/HTML e recuperação da interface Swing.

Classe	Testes na execução final
GaussianOutputParserTest	10
SmilesServiceTest	4
CoordinatesAndInputTest	6
BatchTest	3
ModelAndAnalysisTest	3
ReportTest	2
SwingSmokeTest	1
Total	29

A execução final de mvn -B clean test package terminou com BUILD SUCCESS (código de saída 0), em 30/09/2026, às 23:56:30 -03:00. Foram executados 29 testes, sem falhas, erros ou pulos. O log sanitizado está em docs/validation/maven-build.txt. O JAR target/chemest-java-1.0-SNAPSHOT.jar foi gerado e sua existência foi confirmada.

O smoke test inicia App, verifica cinco abas, valida CCO e confirma recuperação após um SMILES inválido. Em ambiente sem display, esse teste pode ser pulado explicitamente. Não há teste de execução quântica externa nem de todos os diálogos da interface.

10. Limitações

- Somente orbitais Alpha; sem suporte Beta.
- SDF e PubChem não implementados.
- Gaussian e PySCF não são executados pelo sistema.
- O BCF não foi testado dentro do Gaussian.
- XYZ deve ser fornecido pelo usuário; não há otimização ou validação física completa da geometria.
- O OUT demonstrativo é sintético; suas energias não são resultados científicos do etanol fornecido.
- A comparação Python/Java cobre uma amostra controlada e uma referência independente; não comprova equivalência integral com o código do artigo.
- A presença de orbitais não comprova convergência nem término normal do cálculo.
- O JAR de entrega não inclui dependências embutidas; a execução recomendada usa Maven ou IntelliJ.

11. Conclusão

A entrega demonstra preparação de entrada Gaussian, importação e validação de dados, análise dos orbitais e exportação de resultados usando as classes existentes do ChemEST. A referência Python e o parser Java reproduziram as mesmas energias na amostra sintética, com diferenças absolutas nulas.

Os artefatos tornam explícita a separação entre entradas didáticas, saídas do aplicativo e documentação. O código de produção foi preservado; nenhuma funcionalidade científica opcional foi adicionada. A validação é computacional e estrutural, sem alegar execução Gaussian, otimização quântica ou resultados experimentais.

12. Referência

Han, D.; Han, H.; Chen, E.; Cesarski, W. J.; Lim, H.; Shin, H.; Cowart, S. V.; Nagelli, E. A.; Yuk, S. F.; Jeong, K.

Integrating Electronic Structure Theory Calculations with Python and Cheminformatics Tools.

Journal of Chemical Education, 2025, 102, 4115-4122.

DOI: <https://doi.org/10.1021/acs.jchemed.4c01171>.